

Technical Challenge:

Document Processing System

Project Description

You must build a **Document Processing System** that allows loading, processing, and analyzing text files asynchronously, with a REST API to control and monitor the process.

Required Functionalities

1. Process Control API

- **POST /process/start** - Start a new analysis process
- **POST /process/stop/{process_id}** - Stop a specific process
- **GET /process/status/{process_id}** - Query the status of a process
- **GET /process/list** - List all processes and their states
- **GET /process/results/{process_id}** - Get analysis results

2. Processing System

- Load text files from a specific folder
- Process documents in batches (batch processing)
- Extract basic statistics: word count, lines, characters
- Identify most frequent words
- Generate a content summary

3. System States

- **PENDING** - Process created but not started
- **RUNNING** - Processing in progress
- **PAUSED** - Process temporarily paused
- **COMPLETED** - Process finished successfully
- **FAILED** - Process terminated with errors
- **STOPPED** - Process manually stopped

4. Results Storage

- Persist process states
- Store analysis results
- Maintain activity logs

Technical Specifications

Minimum Requirements

- Functional REST API with specified endpoints
- Real asynchronous processing (not simulated)
- Concurrent state management
- Input validation and error handling
- API documentation

Test Data

You must create at least 10 example text files with varied content (minimum 500 words each) to demonstrate system functionality.

API Response Format

```
{
  "process_id": "uuid-string",
  "status": "RUNNING",
  "progress": {
    "total_files": 10,
    "processed_files": 3,
    "percentage": 30
  },
  "started_at": "2024-01-15T10:30:00Z",
  "estimated_completion": "2024-01-15T10:32:00Z",
  "results": {
    "total_words": 1500,
    "total_lines": 75,
    "most_frequent_words": ["the", "of", "and", "to", "a"],
    "files_processed": ["doc1.txt", "doc2.txt", "doc3.txt"]
  }
}
```

Deliverables

Source Code

- Git repository with meaningful commit history
- Complete and executable source code
- Necessary configuration files

Documentation

- **README.md** explaining:
 - Approach and design decisions made
 - Installation and usage instructions
 - Key findings and technical considerations
 - How to run tests
- **API_DOCS.md** with detailed endpoint specification
- Comments in code for complex logic

Test Data

- Example text files (minimum 10 files)
- Test scripts or Postman/Insomnia collection

AI Usage Policy

The use of AI tools is **ALLOWED** but must be **COMPLETELY TRANSPARENT**:

1. Declare all tools used
2. Include complete history of prompts employed
3. Specify which code was generated/assisted by AI
4. Document modifications made to generated code
5. Justify the use of these tools in each case

Bonus Points (Optional)

- Basic web interface for monitoring
- Project dockerization
- Structured logging
- Implementation of pause/resume processes
- WebSockets for real-time updates

Evaluation Criteria

Category	Weight	What We Evaluate
Architecture & Design	25%	Separation of concerns, design patterns, modularity, scalability
API Implementation	20%	Functional endpoints, correct HTTP codes, validations, consistency
Concurrency Management	20%	Simultaneous processes, states, threading/async, race conditions
Code Quality	15%	SOLID principles, naming conventions, readability, maintainability
Documentation & Testing	10%	Complete README, tests, comments, ease of execution
Error Handling	10%	Robust validations, logs, graceful degradation, edge cases